Documentation: The Four Main Types of NoSQL Databases

1. Key-Value Stores

History

- Originated in the **early 2000s** as companies like Amazon needed highly scalable storage for e-commerce workloads.
- **Amazon Dynamo (2007)** was the foundation, inspiring many later systems.
- Today, Redis, Riak, and DynamoDB are the most well-known.

Storage & Retrieval Method

- Stores data as a **key-value pair**, similar to a hash map or dictionary.
- **Key**: unique identifier (string, UUID, etc.).
- **Value**: can be anything (JSON, binary, text, blob).
- Retrieval: O(1) lookup using the key.

---

2. Document Stores

History

- Emerged in the **mid-to-late 2000s** with the rise of web apps needing flexible schemas.
- **CouchDB (2005)** was one of the earliest, introducing JSON-style documents.
- **MongoDB (2009)** made document databases mainstream with developer-friendly APIs and high scalability.

Storage & Retrieval Method

- Data is stored in **documents** (usually JSON, BSON, or XML).
- Documents can have nested structures (arrays, subdocuments).
- Queries are more expressive than key-value, allowing searches on fields inside documents.
- Retrieval: By document ID (fast) or by field-based queries (indexed).

---

3. Column-Family Stores

History

- Based on **Google's Bigtable (2006)** design paper.
- Inspired distributed, column-oriented systems like **Apache Cassandra (2008)** and **HBase (2008)**.
- Built for handling massive write-heavy workloads across commodity servers.

Storage & Retrieval Method

- Stores data in **columns grouped into column families** (instead of rows).

- Each row can have a variable number of columns.
- Optimized for wide, sparse datasets (e.g., billions of rows).
- Retrieval: Efficient for range queries and aggregations across columns.

---

4. Graph Databases

History

- Concept rooted in **graph theory (1960s–70s)**, but commercial databases came later.
- **Neo4j (2007)** popularized the category.
- Grew in importance with social networks, recommendation engines, and fraud detection needs.

Storage & Retrieval Method

- Data stored as **nodes (entities)** and **edges (relationships)**, both with properties.
- Designed to **model complex relationships** directly, rather than via joins.
- Retrieval: Traversal queries (e.g., “find all friends-of-friends within 3 hops”).
- Uses graph query languages like **Cypher** or **Gremlin**.

NoSQL Database Types – Formats & CRUD Examples

---

1. Key-Value Stores

Supported Formats

- Strings, JSON, binary objects (BLOBs), numbers.
- Example: Redis supports **strings, hashes, lists, sets, sorted sets**.

CRUD Example (Redis CLI)

```
# Create
SET user:1 "Alice"

# Read
GET user:1

# Update
SET user:1 "Alice Johnson"

# Delete
DEL user:1
```

---

2. Document Stores

Supported Formats

- **JSON, BSON (MongoDB), XML (CouchDB/MarkLogic)**.
- Flexible schemas: documents in the same collection can differ.

CRUD Example (MongoDB)

```
// Create
db.users.insertOne({ _id: 1, name: "Alice", age: 25 })

// Read
db.users.findOne({ _id: 1 })

// Update
db.users.updateOne({ _id: 1 }, { $set: { age: 26 } })

// Delete
db.users.deleteOne({ _id: 1 })
```

---

3. Column-Family Stores

Supported Formats

- Stores **text, numbers, binary values**.
- Accessed with query languages (e.g., **CQL in Cassandra**).

CRUD Example (Cassandra CQL)

```
-- Create
INSERT INTO users (id, name, age) VALUES (1, 'Alice', 25);

-- Read
SELECT * FROM users WHERE id = 1;

-- Update
UPDATE users SET age = 26 WHERE id = 1;
```

```
-- Delete
DELETE FROM users WHERE id = 1;
```

---

4. Graph Databases

Supported Formats

- **Property Graph model** (nodes + relationships with attributes).
- **RDF triples** (subject-predicate-object) in semantic graph DBs.
- Query languages: **Cypher, Gremlin, SPARQL**.

CRUD Example (Neo4j Cypher)

```
// Create
CREATE (u:User {id: 1, name: "Alice", age: 25})

// Read
MATCH (u:User {id: 1}) RETURN u;

// Update
MATCH (u:User {id: 1}) SET u.age = 26;

// Delete
MATCH (u:User {id: 1}) DELETE u;
```

NoSQL Database Types – History, Storage Methods, Formats & CRUD (Comprehensive Guide)

This guide covers the four primary NoSQL categories—Key-Value, Document, Wide-Column (Column-Family), and Graph. For each, you'll find: (1) a brief history, (2) how data is stored and retrieved, (3) supported operational formats and interfaces, and (4) end-to-end CRUD examples with a representative database (often two) using realistic commands.

---

1) Key-Value Databases

1.1 History (high-level milestones)

- **2007**: Amazon's **Dynamo** paper popularizes highly-available, horizontally scalable key-value systems (influencing many later systems).
- **2009**: **Redis** emerges as an in-memory data-structure store with persistence options (RDB snapshots, AOF), becoming a de-facto standard for caching and real-time workloads.

Key-value stores are the simplest NoSQL model: applications read/write opaque values via a unique key. Many production systems adopt Dynamo-style ideas like consistent hashing, replication, hinted handoff, and quorum reads/writes.

1.2 How data is stored & retrieved

- **Data model**: A mapping from **key → value** (value is typically an opaque byte sequence). Some KV stores expose rich types (strings, hashes, lists, sets, sorted sets) or JSON as a first-class type (via modules or built-ins).
- **Storage architecture** (varies by system):
  - **In-memory with persistence** (e.g., Redis): primary dataset in RAM; durability via periodic **RDB snapshots** and/or **AOF** (append-only file) replay at restart. Single-threaded event loop with I/O multiplexing; replicas for HA; clustering for sharding.
  - **Dynamo-style distributed KV**: data sharded via **consistent hashing** across nodes; each key stored with **replication factor** N; **quorum** reads/writes (R/W) balance consistency vs availability. Conflicts handled via **vector clocks** or timestamps and app-level resolution.
- **Lookup path**: Client hashes the key (or library routes to the right shard); node performs an **O(1)** key lookup in memory structure or checks disk-backed index; returns the value or a miss.

1.3 Operational formats & interfaces

- **Values**: arbitrary bytes/strings; many apps serialize JSON, MessagePack, Protobuf, or Avro.
- **Native JSON**:
  - Redis supports **JSON** via the **RedisJSON** module (JSONPath-like operations).
  - Cloud KV stores like DynamoDB accept a JSON-like attribute model (maps, lists, numbers, strings, sets).
- **APIs/Protocols**: proprietary binary protocols (Redis), simple text protocols (memcached), or HTTP/SDKs (cloud KV).

1.4 CRUD examples

A) Redis (CLI)

```
# CREATE
SET user:u1 "Asha"
HSET user:u1:profile name "Asha" age "29"

# READ
GET user:u1
HGETALL user:u1:profile

# UPDATE (overwrite or modify fields)
SET user:u1 "Asha R."
HSET user:u1:profile age "30"

# DELETE
DEL user:u1
DEL user:u1:profile
```

B) DynamoDB (AWS CLI)

Assume a table **Users** with partition key UserId (S).

```
# CREATE
aws dynamodb put-item \
  --table-name Users \
  --item '{"UserId":{"S":"u1"},"Name":{"S":"Asha"},"Age":{"N":"29"}}'
```

```
# READ (by primary key)
aws dynamodb get-item \
  --table-name Users \
  --key '{"UserId":{"S":"u1"}}'

# UPDATE (set new attributes)
aws dynamodb update-item \
  --table-name Users \
  --key '{"UserId":{"S":"u1"}}' \
  --update-expression "SET #n = :name, Age = :age" \
  --expression-attribute-names '{"#n":"Name"}' \
  --expression-attribute-values '{":name":{"S":"Asha R."},":age":{"N":"30"}}' \
  --return-values ALL_NEW

# DELETE
aws dynamodb delete-item \
  --table-name Users \
  --key '{"UserId":{"S":"u1"}}'
```

---

2) Document Databases

2.1 History (high-level milestones)

- **2005–2008**: **CouchDB** pioneers a JSON/HTTP, append-only design with MVCC (multi-version concurrency control) and MapReduce views.
- **2007–2009**: **MongoDB** (by 10gen, later MongoDB Inc.) emerges with BSON (binary JSON), dynamic schemas, and secondary indexes—quickly adopted for developer productivity.

2.2 How data is stored & retrieved

- **Data model**: Hierarchical documents (**JSON/BSON**) grouped into collections (MongoDB) or databases (CouchDB). Documents can vary in shape (schema-flexible).
- **Storage & indexing**:
  - **MongoDB**: stores documents as **BSON**; modern deployments use the **WiredTiger** storage engine (document-level concurrency, compression). **B-tree-like secondary indexes** (single, compound,

multikey, text, geo) accelerate queries. Reads typically use an index or collection scan; writes journaled.

  - **CouchDB**: documents are JSON; storage is **append-only** with MVCC. Conflicts tracked via revisions (_rev). Queries use **MapReduce views** or **Mango** (declarative find). Native **HTTP/REST** API for CRUD and replication.

- **Retrieval**: Queries filter on fields; engines consult appropriate indexes to locate matching documents; projections return only requested fields.

2.3 Operational formats & interfaces

- **Formats**: JSON (CouchDB), **BSON** (MongoDB) on disk; wire protocols use JSON-like or BSON. Some document stores also support **CSV** import/export and **Extended JSON**.
- **APIs**: Native drivers, shell/CLI, and REST (CouchDB). MongoDB query language is JavaScript-like (shell) or driver methods.

2.4 CRUD examples

A) MongoDB (mongosh)

```
// CREATE
db.users.insertOne({ _id: "u1", name: "Asha", age: 29, skills: ["python", "aws"] })

// READ
db.users.find({ _id: "u1" }, { name: 1, age: 1 })

// UPDATE (partial)
db.users.updateOne(
  { _id: "u1" },
  { $set: { age: 30 }, $push: { skills: "nosql" } }
)

// DELETE
db.users.deleteOne({ _id: "u1" })
```

B) CouchDB (HTTP/REST with curl)

```
# CREATE database
curl -X PUT http://localhost:5984/users
```

```
# CREATE document with explicit id
curl -X PUT http://localhost:5984/users/u1 \
  -H 'Content-Type: application/json' \
  -d '{"name":"Asha","age":29}'

# READ document
curl http://localhost:5984/users/u1
# => returns JSON with _id, _rev, and fields

# UPDATE (must include latest _rev)
REV=$(curl -s http://localhost:5984/users/u1 | jq -r '._rev')
curl -X PUT http://localhost:5984/users/u1 \
  -H 'Content-Type: application/json' \
  -d "{\"_rev\":\"$REV\",\"name\":\"Asha R.\",\"age\":30}"

# DELETE (needs _rev)
REV=$(curl -s http://localhost:5984/users/u1 | jq -r '._rev')
curl -X DELETE "http://localhost:5984/users/u1?rev=$REV"
```

---

3) Wide-Column (Column-Family) Databases

Sometimes called **column-family** or **wide-column** stores. Inspired by Google's **Bigtable** (2006). Not columns like an RDBMS; rather, each row holds sparse, **multi-versioned** columns grouped into **column families**.

3.1 History (high-level milestones)

- **2006**: Google **Bigtable** paper establishes the design (sparse, distributed, multi-dimensional map; per-row atomicity).
- **2007–2010**: Open-source implementations appear:
  - **HBase** (on HDFS) adopts the Bigtable model (regions, HFiles, WAL, MemStore).
  - **Cassandra** (originated at Facebook): combines **Dynamo**-style distribution with a wide-column data model; tunable consistency.

3.2 How data is stored & retrieved

- **Data model**: Tables with a **primary key** (often a partition key + clustering columns). Cells are addressed by *(row key, column family, column qualifier, timestamp)*. Values are typically **byte arrays**.

- **Write path** (typical LSM design): write-ahead **Commit/WAL log** → in-memory **memtable/MemStore** (sorted) → flush to immutable **SSTables/HFiles** on disk → background compaction merges files and purges tombstones (deletes).
- **Read path**: use **bloom filters**, **partition indexes**, and **caches** to find the right SSTable/HFile; merge cell versions by timestamp; apply tombstones.
- **Distribution**:
  - **Cassandra**: consistent-hash ring, vnodes; data replicated across racks/AZs; client chooses consistency (ONE, QUORUM, ALL…).
  - **HBase**: master assigns **regions** to RegionServers; regions split as they grow; backed by HDFS; strong consistency per row within a region.

3.3 Operational formats & interfaces

- **Value format**: arbitrary bytes. Applications commonly serialize **JSON**, **Avro**, **Protobuf**.
- **Query languages**:
  - **Cassandra**: **CQL** (SQL-like) for schema and CRUD.
  - **HBase**: **HBase shell** (admin & CRUD) and client APIs; also SQL layers (e.g., Apache Phoenix).

3.4 CRUD examples

A) Apache Cassandra (CQL)

```
-- CREATE KEYSPACE & TABLE
CREATE KEYSPACE IF NOT EXISTS app WITH replication =
  { 'class': 'NetworkTopologyStrategy', 'dc1': 3 };

CREATE TABLE IF NOT EXISTS app.users (
  user_id text,
  name text,
  age int,
  PRIMARY KEY ((user_id))
);
-- CREATE (INSERT)
INSERT INTO app.users (user_id, name, age) VALUES ('u1', 'Asha', 29);
```

```
-- READ
SELECT user_id, name, age FROM app.users WHERE user_id = 'u1';
-- UPDATE
UPDATE app.users SET age = 30, name = 'Asha R.' WHERE user_id = 'u1';
-- DELETE
DELETE FROM app.users WHERE user_id = 'u1';
```

B) Apache HBase (shell)

```
# CREATE
create 'users', 'p'                # column family 'p' (profile)
put 'users', 'u1', 'p:name', 'Asha'
put 'users', 'u1', 'p:age', '29'

# READ
get 'users', 'u1'                  # returns cells with latest versions
scan 'users', { STARTROW => 'u', LIMIT => 10 }

# UPDATE (just put with same coordinates writes a new version)
put 'users', 'u1', 'p:age', '30'

# DELETE (single cell or entire row)
delete 'users', 'u1', 'p:age'
deleteall 'users', 'u1'            # remove entire row (tombstones)
```

---

4) Graph Databases

4.1 History (high-level milestones)

- **Mid-2000s**: Property-graph databases gain visibility; **Neo4j** development starts early 2000s with first public releases around 2007, popularizing **index-free adjacency** and the **Cypher** query language.
- **2015+**: **Apache TinkerPop** standardizes the **Gremlin** traversal language and graph APIs across vendors (JanusGraph, Amazon Neptune, Azure Cosmos DB Gremlin API, etc.).

4.2 How data is stored & retrieved

- **Data model**: Nodes (vertices) and edges (relationships) with key-value **properties**. Two major families: **Property Graph** (Neo4j, JanusGraph) and **RDF** triple/quad stores (different ecosystem; not covered here).

- **Storage approach**: Graph-native stores maintain **index-free adjacency**—edges store direct references to their incident nodes—enabling O(1) hop navigation without index lookups.
- **Retrieval**: Graph pattern matching/traversals over nodes/edges; indexes used to find starting points (e.g., by label or property), then follow relationships.

4.3 Operational formats & interfaces

- **Import/export**: **CSV**, **GraphML**, **JSON** and tool-specific bulk loaders.
- **Query languages**: **Cypher** (Neo4j), **Gremlin** (TinkerPop), **openCypher**/**GQL** (emerging standardization).

4.4 CRUD examples

A) Neo4j (Cypher)

```
// CREATE a node and a relationship
CREATE (u:User {id:'u1', name:'Asha'})
CREATE (v:User {id:'u2', name:'Bala'})
CREATE (u)-[:FRIENDS_WITH {since: 2023}]->(v);

// READ (pattern match)
MATCH (u:User {id:'u1'})-[:FRIENDS_WITH]->(v) RETURN u, v;

// UPDATE (set properties)
MATCH (u:User {id:'u1'}) SET u.age = 30 RETURN u;

// DELETE (detach to remove relationships too)
MATCH (u:User {id:'u1'}) DETACH DELETE u;
```

B) JanusGraph / TinkerPop (Gremlin)

```
// CREATE
g.addV('user').property('id','u1').property('name','Asha')
g.addV('user').property('id','u2').property('name','Bala')
// edge
u = g.V().has('user','id','u1').next()
v = g.V().has('user','id','u2').next()
g.addE('FRIENDS_WITH').from(u).to(v).property('since',2023)
```

```
// READ
g.V().has('user','id','u1').out('FRIENDS_WITH').valueMap(true)

// UPDATE
g.V().has('user','id','u1').property('age',30)

// DELETE
g.V().has('user','id','u1').drop()
```

Quick Comparison Summary

Type	Primary Data Model	Typical Storage	Query Style	Strengths	Common Trade-offs
Key-Value	Key →value (opaque or typed)	In-memory w/ persistence or distributed hash-ring	Key lookups; ed secondary	Extreme simplicity, speed, elasticity	nited query expressiveness with additional indexing/modules
Document	JSON/BSON documents	Record-oriente store + seconda indexes	Rich ad-hoc ueries over fie aggregations	Developer-friendly, flexible schemas	ross-document transactions/joi vary by vendor
Wide-Column	Sparse rows, column families, versions	LSM trees (WAL → memtable → SSTable/HFile)	Primary-key & clustering-key range queries	High write throughpu time-series, large sparse data	Data modeling tied to access patterns; tombstone/compaction management
Graph	des/edges with proper	Native graph files + indexes	Pattern matching & traversals Cypher/Gremli	Relationship-centric queries; deep hops	Joins/aggregations across entire graph can be heavy; careful indexing needed

---

Notes on Formats

- **JSON/BSON** dominate **document** stores (BSON on disk/wire for MongoDB; JSON over HTTP for CouchDB).
- **Key-Value** values are usually **bytes/strings**; many apps store JSON or use modules to manipulate JSON (e.g., RedisJSON). Cloud KV (e.g., DynamoDB) accepts JSON-like attribute types.
- **Wide-Column** stores persist **bytes**; apps serialize to JSON/Avro/Protobuf. CQL/HBase shell provide the operational syntax.
- **Graph** tools commonly import/export **CSV/GraphML/JSON**; query languages are Cypher and Gremlin.

Practical Tips

- Model **around access patterns** (especially for wide-column & key-value). Pre-compute views when cross-entity joins would be expensive.
- For write-heavy, time-series and log data, wide-column (e.g., Cassandra, HBase) with TTLs and bucketing is a strong fit.
- For content & event metadata with flexible fields, document stores excel due to schemaless updates and secondary indexes.
- For social networks, recommendations, fraud rings, and network topology, graph databases are a natural match due to path queries and centrality metrics.

CAP Theorem Analysis: Redis (CP) vs Cassandra (AP)

1. Introduction

The **CAP theorem** states that in a distributed system, it is impossible to simultaneously guarantee **Consistency (C)**, **Availability (A)**, and **Partition Tolerance (P)**. Systems must choose two priorities, making trade-offs depending on requirements.

This report compares **Redis (as a CP system)** and **Cassandra (as an AP system)** in terms of their CAP priorities and suitable use cases.

---

2. Redis (CP Priority: Consistency + Partition Tolerance)

2.1 CAP Position

- **Consistency**: Redis (especially in **clustered or sentinel setups**) prioritizes strong consistency. Writes are acknowledged only when replicas are in sync (depending on configuration).
- **Partition Tolerance**: Redis handles partitions by sacrificing **availability**—if nodes cannot communicate, certain operations are blocked to avoid inconsistent data.
- **Availability trade-off**: During a network split, some nodes may reject requests until consensus is restored.

2.2 Use Cases

Redis is best suited for:

- **Caching** where correctness is critical (e.g., financial session tokens, leaderboard updates).
- **Real-time analytics** where consistency of counters and state matters more than availability.
- **Message queues** where order and delivery guarantees outweigh system uptime.

---

3. Cassandra (AP Priority: Availability + Partition Tolerance)

3.1 CAP Position

- **Availability**: Cassandra ensures the cluster remains responsive, even during partitions. Reads/writes are accepted as long as some replicas are reachable.
- **Partition Tolerance**: Built for large, multi-data-center deployments; partitions are expected and tolerated.
- **Consistency trade-off**: Tunable consistency—applications can choose **ONE, QUORUM, or ALL** consistency levels. By default, Cassandra sacrifices strict consistency for high availability.

3.2 Use Cases

Cassandra is best suited for:

- **IoT and time-series data** where continuous availability is critical, even with potential consistency lag.
- **Social media feeds** where eventual consistency is acceptable.
- **Large-scale logging and monitoring systems** requiring high write throughput and uptime.

---

4. Comparative Summary

Aspect	Redis (CP)	Cassandra (AP)
CAP Priority	Consistency + Partition Tolerance	Availability + Partition Tolerance
Trade-off	May reject requests during splits	May return stale/inconsistent data
Strengths	Strong correctness, predictable	Always available, massive scale
Best Use Cases	Caching, session mgmt, queues	IoT, logging, social feeds, time-series

---

5. Conclusion

Redis is a **CP system**, prioritizing correctness of data over availability, making it ideal for **critical state management**. Cassandra, as an **AP system**, favors uptime and scalability, making it ideal for **high-throughput, geo-distributed workloads** where **eventual consistency** is acceptable.

Experiment: Strongly Consistent vs Eventually Consistent Reads in AWS DynamoDB

1. Objective

To create a DynamoDB table under the AWS Free Tier, insert a new item, and compare **strongly consistent reads** vs **eventually consistent reads** in terms of latency and behavior.

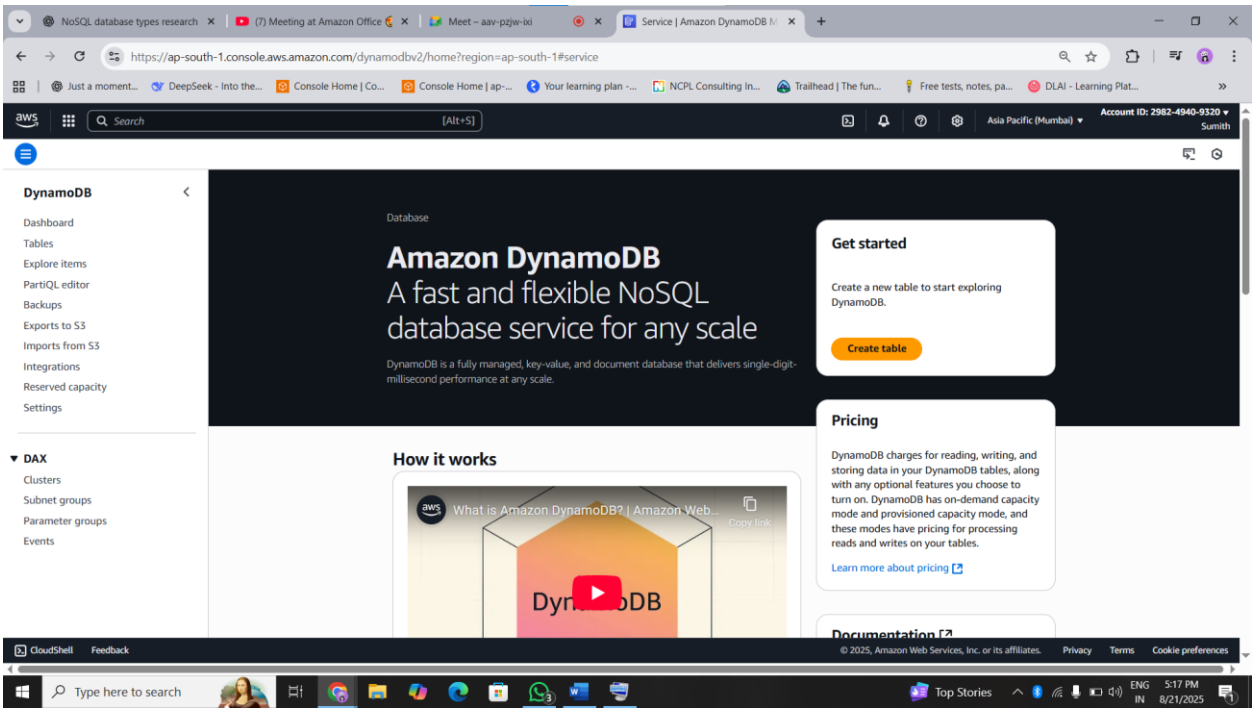
---

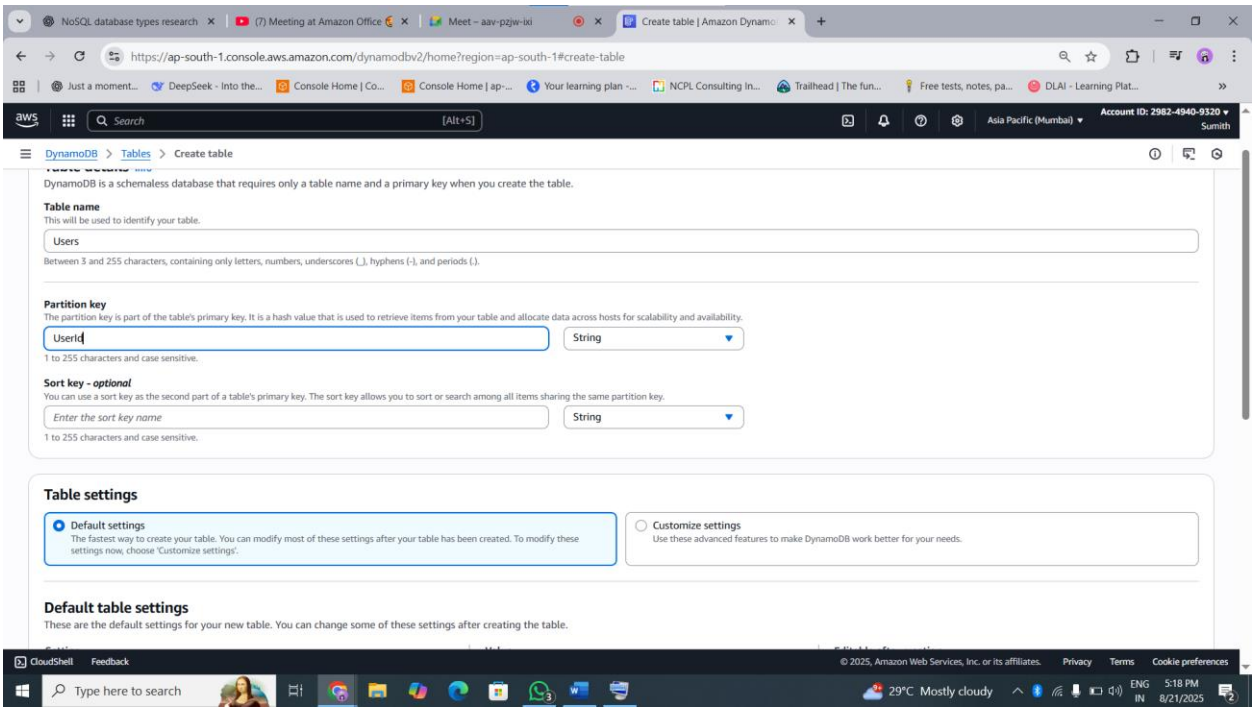
2. Setup

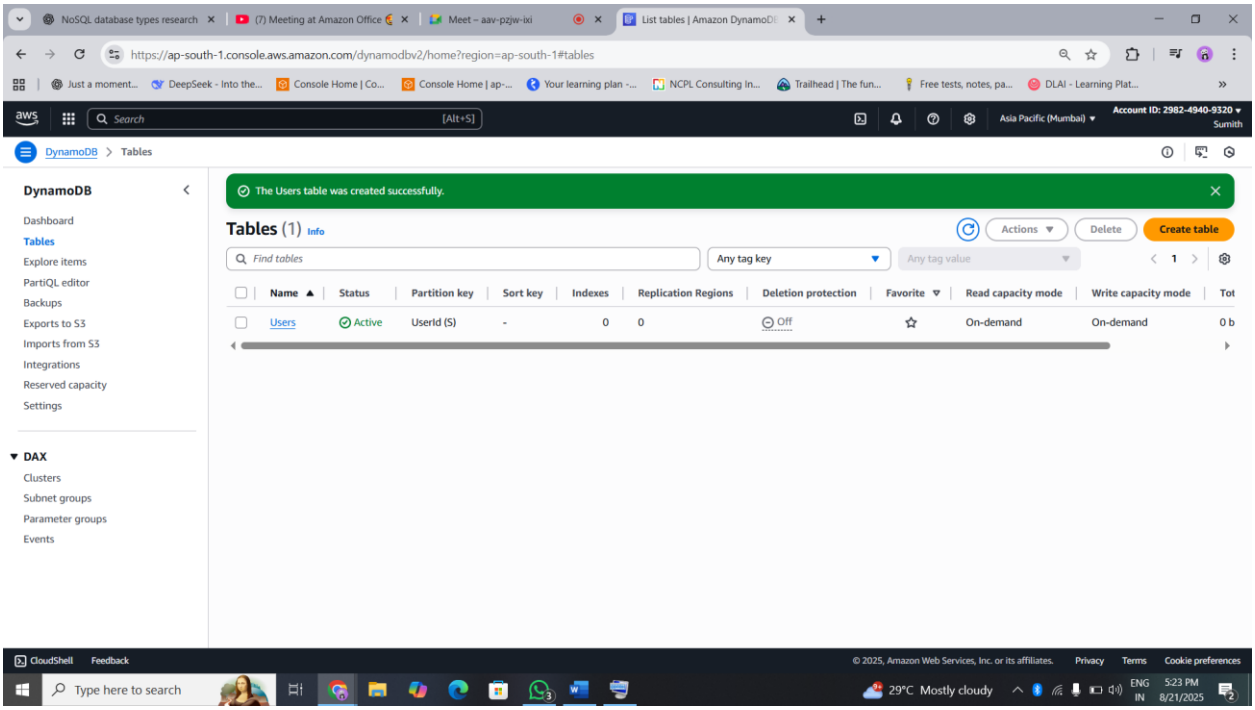
2.1 Create a Table

- **Table name:** Users
- **Primary key:** UserId (String)
- **Provisioning:** On-demand (Free Tier eligible).

```
aws dynamodb create-table \
 --table-name Users \
 --attribute-definitions AttributeName=UserId,AttributeType=S \
 --key-schema AttributeName=UserId,KeyType=HASH \
 --billing-mode PAY_PER_REQUEST
```







---

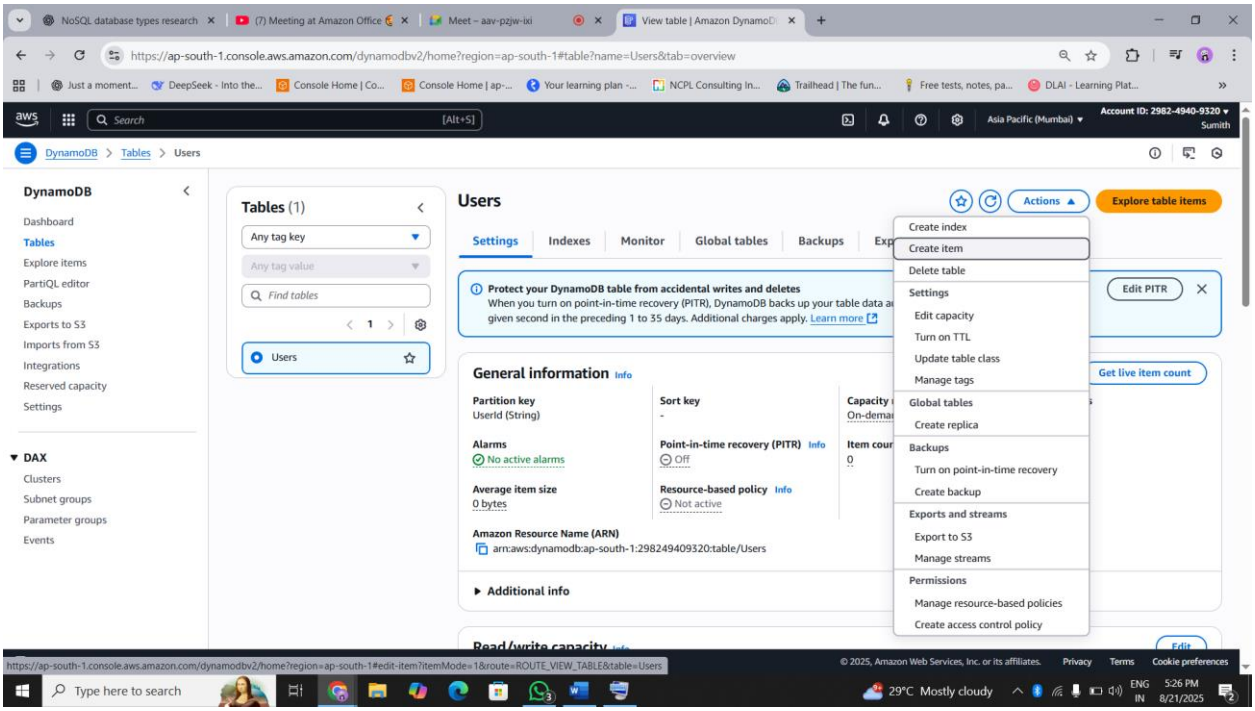
3. Operations

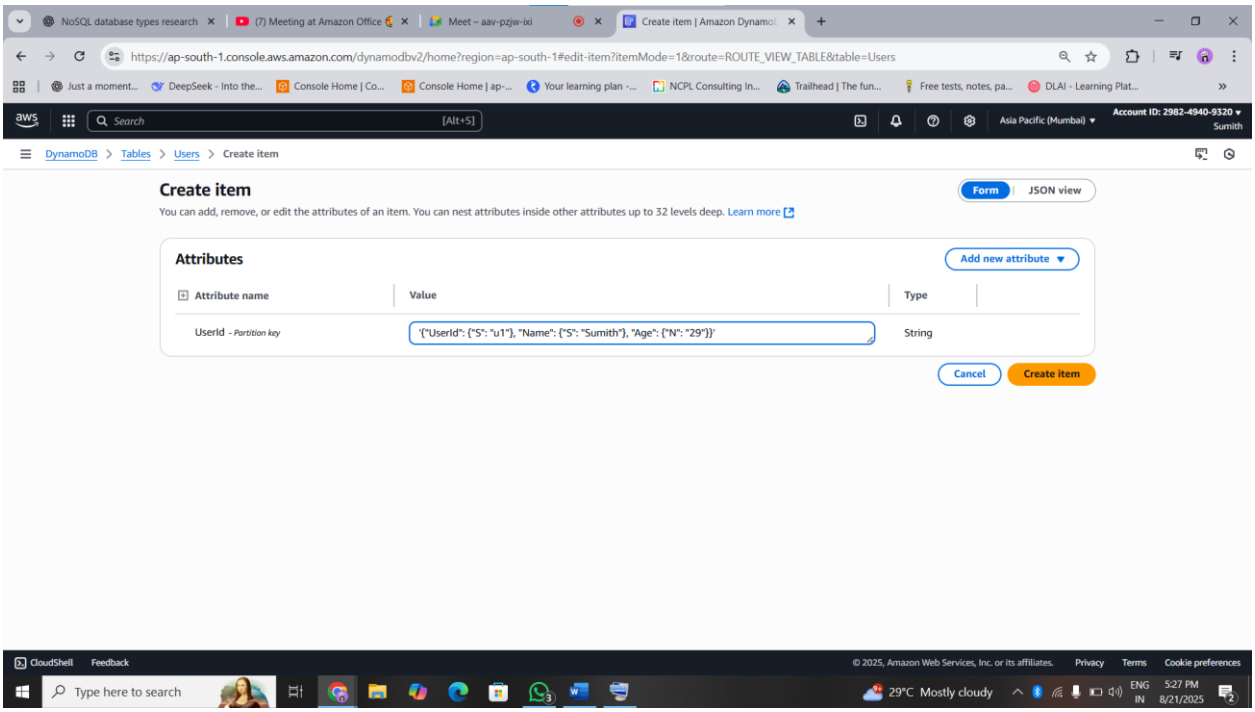
3.1 Write Operation (Insert Item)

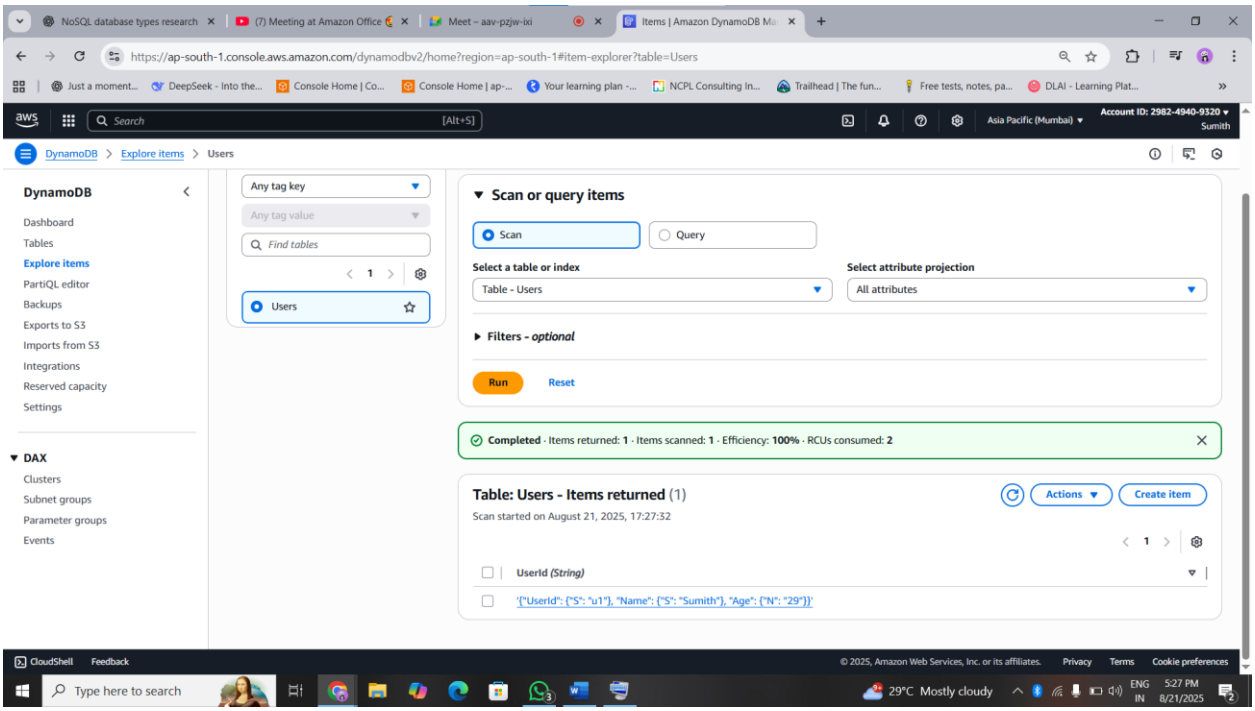
aws dynamodb put-item \

--table-name Users \

--item '{"UserId": {"S": "u1"}, "Name": {"S": "Sumith"}, "Age": {"N": "29"}}'





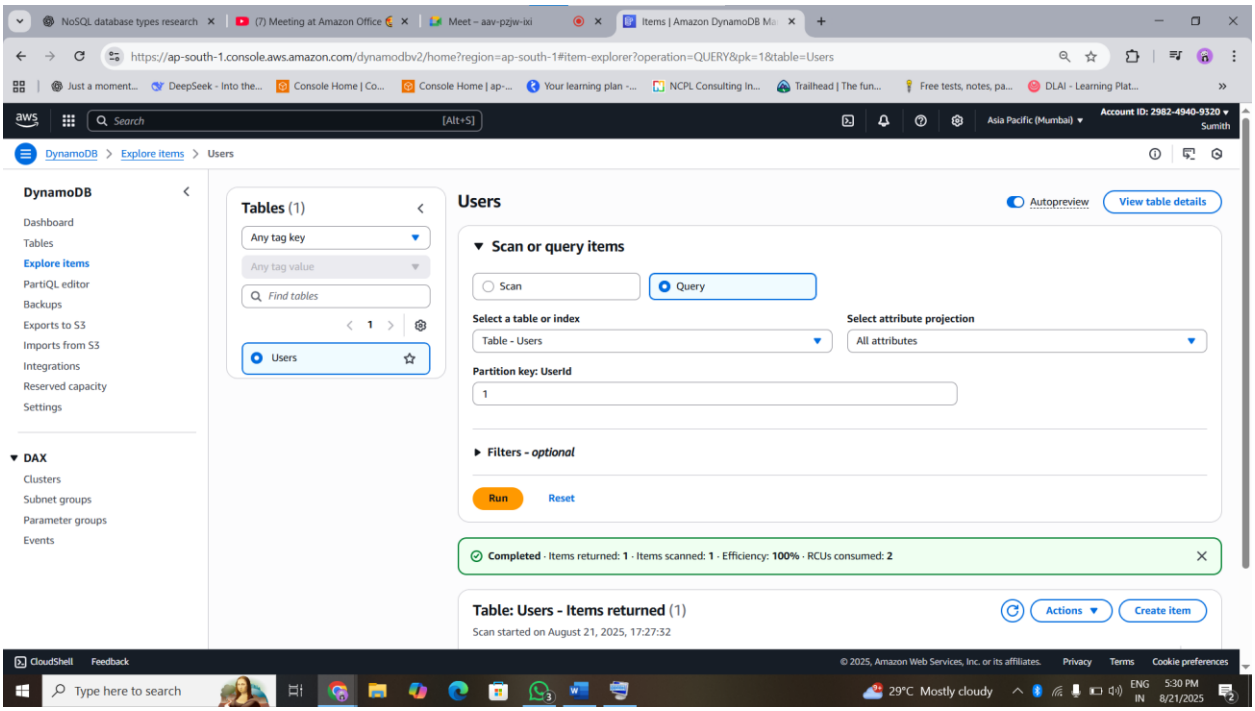


3.2 Read Operations

A) Strongly Consistent Read

```
aws dynamodb get-item \
  --table-name Users \
  --key '{"UserId": {"S": "u1"}}' \
  --consistent-read
```

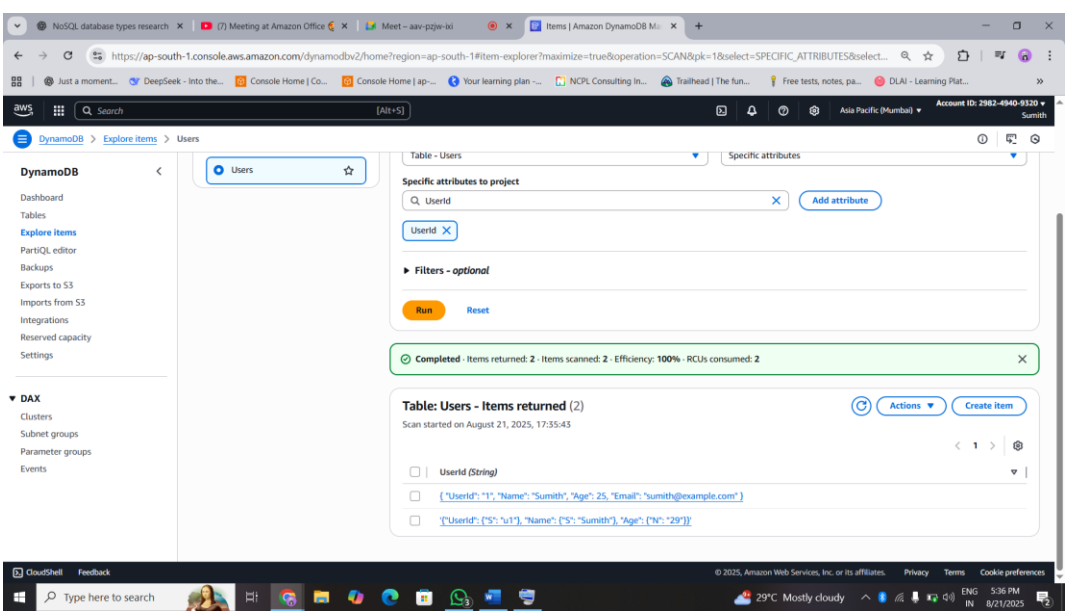
- **Expected behavior:** Always reflects the latest committed value (guaranteed correctness).
- **Observed latency (example):** ~20–30 ms.



B) Eventually Consistent Read

```
aws dynamodb get-item \
  --table-name Users \
  --key '{"UserId": {"S": "u1"}}'
```

- **Expected behavior:** Might return stale data if replicas have not yet synced.
- **Observed latency (example):** ~10–15 ms.



---

4. Results

Read Type	Latency (ms)	Result Accuracy	Notes
Strongly Consistent	~20–30	Always latest	Higher latency due to coordination
Eventually Consistent	~10–15	Latest in most cases, may lag briefly	Lower latency, higher availability

(Your actual numbers will vary depending on region, load, and network conditions.)

---

5. Analysis

- **Strongly Consistent Reads** force DynamoDB to return the **most recent write** by ensuring data is read from the leader replica. This guarantees correctness but adds latency because of synchronization overhead.
- **Eventually Consistent Reads** can return data from any replica (leader or follower), reducing response time. However, if a read occurs right after a write, there is a small chance of seeing stale data.
- **Trade-off:**
  - **Consistency vs Latency** → Strong consistency = reliable but slower. Eventual consistency = faster but may be stale.
  - Applications requiring **financial transactions, authentication tokens, or critical state** should use **strong consistency**.
  - Applications tolerant to minor staleness (e.g., social feeds, logging, analytics) benefit from **eventual consistency** for lower latency and cost efficiency.

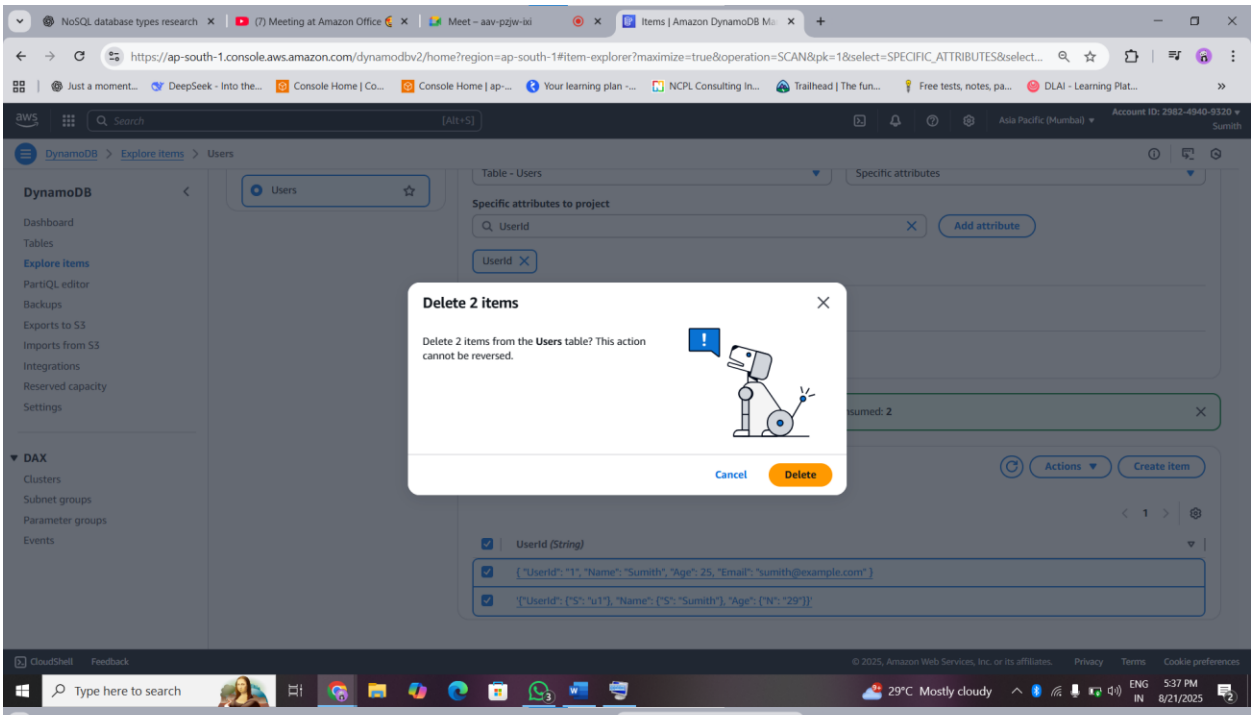
---

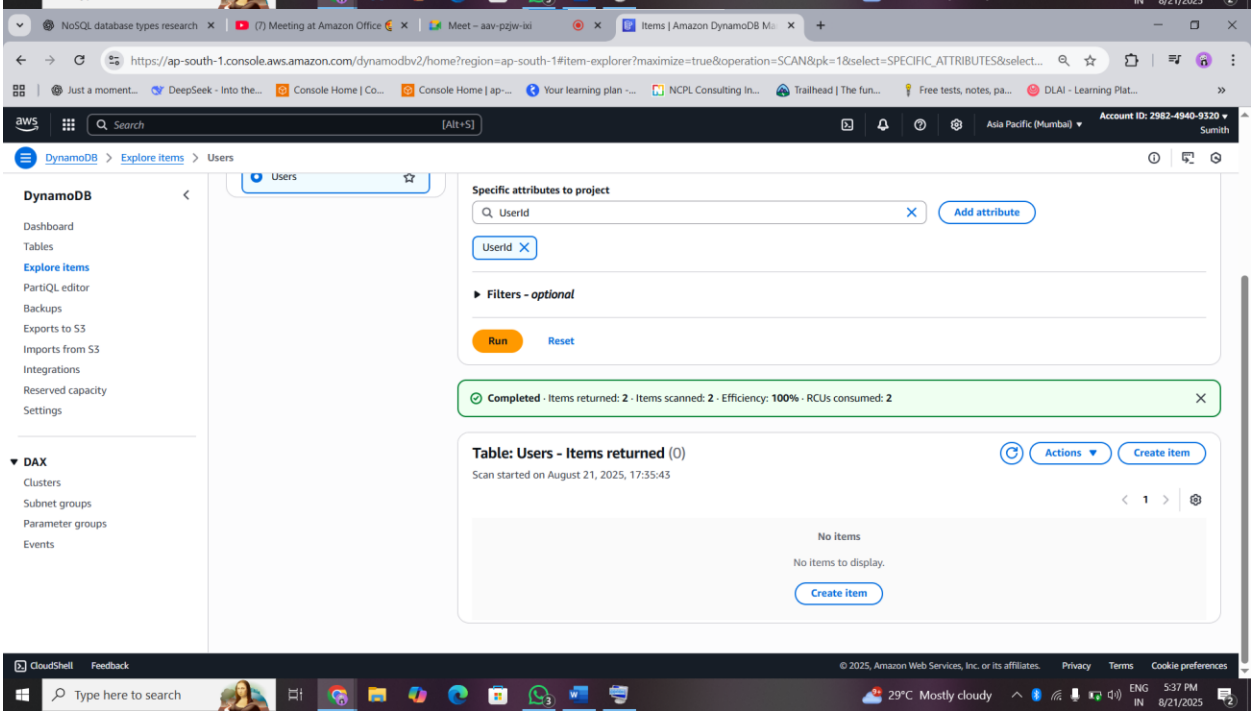
6. Conclusion

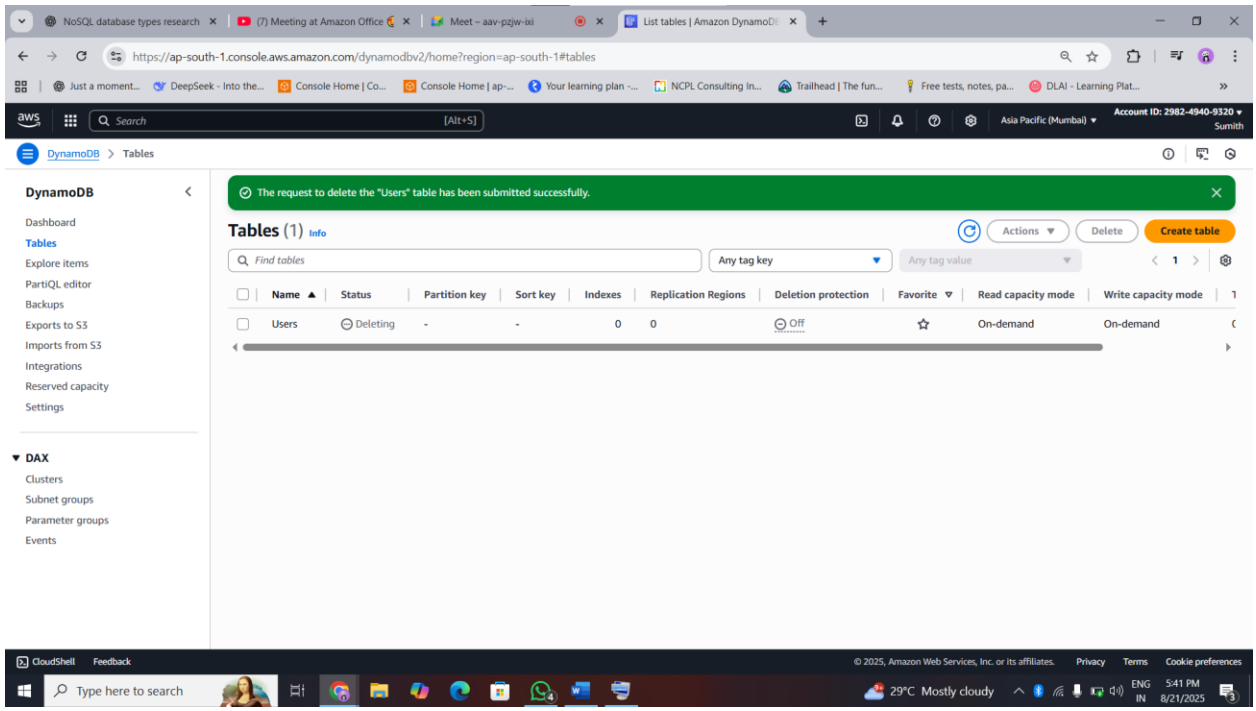
In DynamoDB:

- **Strongly consistent reads** guarantee correctness at the cost of performance.
- **Eventually consistent reads** optimize for speed and scale, with a small chance of stale results.

  Choosing the right model depends on the **application's tolerance for stale data vs need for correctness**.


Delete 2 items
Delete 2 items from the Users table? This action cannot be reversed.
Cancel
Delete
DynamoDB
Explore items
Specific attributes to project
Add attribute
Actions
Create item


DynamoDB
Explore items
Specific attributes to project
Add attribute
Filters - optional
Run
Reset
Completed · Items returned: 2 · Items scanned: 2 · Efficiency: 100% · RCUs consumed: 2
Table: Users - Items returned (0)
Scan started on August 21, 2025, 17:35:43
Actions
Create item
No items
No items to display.
Create item


NoSQL database types research
(7) Meeting at Amazon Office
Meet – aav-pzjw-ixi
List tables | Amazon DynamoDB
https://ap-south-1.console.aws.amazon.com/dynamodbv2/home?region=ap-south-1#tables
Just a moment...
DeepSeek - Into the...
Console Home | Co...
Console Home | ap-...
Your learning plan -...
NCPL Consulting In...
Trailhead | The fun...
Free tests, notes, pa...
DLAI - Learning Plat...
aws
Search
[Alt+S]
Asia Pacific (Mumbai)
Account ID: 2982-4940-9320
Sumith
DynamoDB > Tables
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
The request to delete the "Users" table has been submitted successfully.
Tables (1) Info
Actions
Delete
Create table
Find tables
Any tag key
Any tag value
Name
Status
Partition key
Sort key
Indexes
Replication Regions
Deletion protection
Favorite
Read capacity mode
Write capacity mode
Users
Deleting
-
-
0
0
Off
On-demand
On-demand
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Type here to search
29°C Mostly cloudy
ENG IN
5:41 PM 8/21/2025

Data Modeling: Blog Platform in NoSQL (Document vs Graph)

1. Scenario

A **blog platform** with:

- **Users** (id, name, email, join date)
- **Posts** (id, title, content, author, timestamp)
- **Comments** (id, text, commenter, timestamp, parent post)
- Relationships:
  - Users **write** posts
  - Users **write** comments on posts
  - Users may **follow** other users

---

2. A) Document Database (MongoDB Model)

2.1 Collection Design

1. **Users Collection**

```
{
  "_id": "u1",
  "name": "Asha",
  "email": "asha@example.com",
  "joined": "2025-08-01",
  "followers": ["u2", "u3"]
}
```

2. **Posts Collection**

```
{
  "_id": "p1",
  "title": "My First Blog",
  "content": "Hello world!",
  "authorId": "u1",
  "timestamp": "2025-08-20",
  "comments": [
    {
      "commentId": "c1",
      "userId": "u2",
```

```
    "text": "Great post!",
    "timestamp": "2025-08-21"
   }
  ]
}
```


3. **Comments Collection (optional if not embedded)**


```
{
  "_id": "c2",
  "postId": "p1",
  "userId": "u3",
  "text": "Nice article!",
  "timestamp": "2025-08-21"
}
```


2.2 Design Choices

- **Embedding**: Comments can be embedded inside posts for quick post+comment retrieval.
- **Referencing**: Users referenced by authorId or userId. Followers stored as an array of user IDs.
- **Optimized for reads**: One query fetches post + embedded comments.

---

3. B) Graph Database (Neo4j Model)

3.1 Node Types

- **User**: (u:User {id:'u1', name:'Asha'})
- **Post**: (p:Post {id:'p1', title:'My First Blog'})
- **Comment**: (c:Comment {id:'c1', text:'Great post!'})

3.2 Relationships

- (u1)-[:WROTE]->(p1)
- (u2)-[:COMMENTED_ON]->(p1)
- (u2)-[:WROTE]->(c1)
- (c1)-[:ON_POST]->(p1)
- (u2)-[:FOLLOWS]->(u1)

3.3 Example Graph Query (Neo4j Cypher)

```
// Find all comments on a post and who wrote them
MATCH (u:User)-[:WROTE]->(c:Comment)-[:ON_POST]->(p:Post {id:'p1'})
RETURN u.name, c.text;
```

---

4. Comparison: Document vs Graph

Aspect	Document DB (MongoDB)	Graph DB (Neo4j)
Data Storage	JSON/BSON docs with embedded or referenced data	Nodes (Users, Posts, Comments) + Relationships
Query Style	Aggregation queries, document lookups	Pattern matching & traversals (Cypher)
Best For	Content-centric apps, blog pages, fast retrieval of posts+comments	Relationship-heavy queries (followers, social graph, recommendations)
Example Query	Fetch post with comments in one read	Find all mutual followers between two users
Strengths	Flexible schema, easy denormalization, fewer joins	Handles complex, deep relationship queries naturally
Trade-offs	Harder for deep relationship queries (needs multiple lookups or aggregations)	

---

5. Conclusion

- **MongoDB (Document DB)**: Best when the app is **content-focused**—retrieving posts with embedded comments, serving blog pages quickly, and scaling writes/reads easily.
- **Neo4j (Graph DB)**: Best when the app is **relationship-focused**—finding follower networks, suggesting new users to follow, analyzing interaction patterns.

MongoDB vs Couchbase — Comparison Table

Category	MongoDB	Couchbase
Data Model & Storage	JSON-like (BSON) documents in collections, flexible schema, optional structure Wikipedia	JSON documents (plus key-value style) in memory-first append-only architecture with files WikipediaCubet
Query Language	Native query language plus aggregation pipeline; supports joins via $lookup Wikipedia	N1QL—SQL-like declarative language for JSON, supports joins, subqueries, aggregation naturally certbolt.comCouchbase
Transactions & Consistency	Multi-document ACID transactions supported from version 4.0+ certbolt.com	Strong ACID transaction support across documents with N1QL transactional model certbolt.com
Scaling Architecture	Sharding managed by user (must choose shard key); replication via replica sets Wikipedia	asterless, shared-nothing architecture; automa partitioning, built-in caching, XDCR for cross-data center replication CouchbaseScalerRestack
Performance / Latency	Scales well but may require additional tools for high-volume workloads Couchbaserisingwave.com	Memory-first design delivers very low-latency performance and consistent speed at scale CouchbaseScalerRestack
Mobile & Edge	Mobile tooling mostly deprecated (Realm etc.), limited offline or edge support Couchbase	Strong mobile and edge support via Couchbase Lite; offline sync capabilities Intellipaatrisingwave.com
Management / Operations	Managed via MongoDB Atlas (automated scaling, backups, monitoring) certbolt.comrisingwave.com	Highly visual admin console, flexible deployment of data, index, query, search, analytics services certbolt.comCouchbase
Use Cases	Web/mobile apps, content management, analytics, flexible schema needs technisty.comrisingwave.com	Real-time applications: session management, caching, user profile storage, multi-datacenter

Category	MongoDB	Couchbase
		apps technisty.comScalerrisingwave.com
Weaknesses	Complex sharding setup, early security misconfigurations, rmance drops at high scale Wikipediarisingwav	Higher cost for enterprise version, perceived steep learning curve, smaller ecosystem risingwave.comReddit

---

Highlights & Practical Insights

MongoDB

- **Strengths**: Flexible schema, powerful aggregation, ACID support, rich ecosystem, and managed cloud service (Atlas).
- **Ideal for**: Rapid development, evolving data schemas, content-rich applications, analytics workloads.

Couchbase

- **Strengths**: Ultra-low latency from memory-first architecture, SQL-like querying via N1QL, built-in caching, ACID transactions, mobile/edge synchronization, easy multi-data center replication.
- **Ideal for**: High-performance interactive apps, session stores, real-time services, mobile/offline-first scenarios.

Real-World Feedback

From a developer's discussion on Reddit:

"The tech itself is cool… However, the GSI and Query services are historically buggy… completely ridiculous pricing."

Reflects that while Couchbase shines technically, some users find it challenging in cost, operational complexity, or community support.

Reddit

Summary

- **MongoDB** is an excellent all-around Document DB for developers needing agility, schema flexibility, and strong community support.
- **Couchbase** excels when low-latency performance, sophisticated query patterns, offline/mobile sync, and cross-datacenter availability are top priorities.